Care Connect

Developer Guide — Architecture, Data Model, Edge Functions, Security

Version 3.0 - August 2026 • https://elcareconnect.lovable.app

1. Stack overview

Frontend: React 18, Vite 5, TypeScript 5, Tailwind v3, shadcn/ui, TanStack Query, React Router v6. Backend: Lovable Cloud (managed Postgres with RLS, Deno edge functions, Realtime broadcast, Storage). Voice: Twilio Programmable Voice plus browser WebRTC. AI: Lovable AI Gateway.

1.1 Repository layout

```
src/
  components/          shadcn UI and feature panels (dashboard/)
  data/documentation.ts in-app documentation content
  hooks/               useAuth, useUserRole, useWebRTCCall, useAgentPresence
  integrations/supabase generated client + types (DO NOT EDIT)
  pages/               route components
  lib/                 phone, normalisers, utilities
supabase/
  functions/           Deno edge functions (one folder = one function)
  migrations/          timestamped SQL
scripts/docs/          PDF generators for this guide
public/docs/           generated User and Developer PDFs
```

2. Routing and navigation

React Router v6 with ScrollToTop remounting on pathname change. ProtectedRoute requires a session; AdminRoute additionally requires admin or super_admin via useUserRole, which reads user_roles. Sidebar filtering is cosmetic — the database is the source of truth.

Route	Guard	Purpose
/	public	Sign in.
/landing	public	Marketing splash.
/dashboard	auth	Analytics, Campaigns, Calls, Appointments, Clients.
/supervisor	auth	Live supervision console.
/setup	admin	People, voice, telephony, catalogue, secrets.
/miscellaneous	admin	Notes, transcripts, transfers, leaderboard, shifts.
/campaign-analytics	admin	Campaign reporting.
/call, /voice	auth	WebRTC panel and AI voice interface.
/profile	auth	Profile, password, privacy.
/documentation	auth	In-app docs, role-filtered chapters.

3. Data model

3.1 Core tables

Table	Purpose
profiles	Public user info linked to the auth user.
user_roles	Role assignment; separate table to prevent privilege escalation.
clients	Customer master records with tags and E.164 phone constraint.
call_campaigns / campaign_types	Campaign definitions and templates.
campaign_runs / campaign_jobs	Per-execution and per-target rows.
outbound_calls	One row per dialed call; six composite indexes.
outbound_call_events (+ monthly partitions)	Append-only Twilio status callback log.
call_queue / call_transfers	Live routing primitives.
agent_status / agent_skills / agent_shifts	Presence, capabilities and rota, keyed by user_id.
product_types / knowledge_base	Catalogue and agent answers.
escalation_settings / emergency_alerts	Escalation rules and raised alerts.
call_transcriptions / campaign_recordings	ASR output and recording metadata.
ivr_menu_options / supported_languages	IVR configuration and localisation.
audit_log / error_events / system_health_metrics	Observability.

3.2 Row level security

Every public table is RLS-enabled. The pattern is: create the table, GRANT to the roles the policies allow, enable RLS, then create policies that delegate to security-definer helpers. Agent presence, skills and shifts are scoped by user_id rather than by email match.

```
create or replace function public.has_role(_uid uuid, _role app_role)
returns boolean language sql stable security definer
set search_path = public as $$
  select exists(select 1 from public.user_roles where user_id=_uid and role=_role)
$$;

create policy "Admins manage all" on public.call_campaigns
for all to authenticated using (public.has_role(auth.uid(),'admin'));
```

Partitions (outbound_call_events_YYYY_MM) are RLS-enabled on creation and carry explicit select policies for supervisors and admins. Security-definer routines are not executable by anon, and only role-check helpers remain executable by authenticated; reporting routines are service-role only. Materialised views are not exposed through the API and are read via gated RPCs.

4. Edge functions

Functions live under supabase/functions/ with index.ts as the entrypoint. Shared helpers in _shared/ cover Twilio signature validation, idempotency, rate limiting, audit logging, phone parsing and monitoring.

Function	Auth	Purpose
ai-voice-call	Twilio signature	Initial TwiML for the outbound dialer.
ai-voice-call-status	Twilio signature	Appends every status into outbound_call_events.
ai-voice-call-dtmf*	Twilio signature	Keypad flows including language and appointments.
ai-voice-call-bridge-ivr	Twilio signature	Connects callers to the configured IVR menu.
process-call-events	CRON_SECRET	Drains the event log into outbound_calls in batches.
campaign-scheduler	Staff JWT	Plans campaign runs inside dialing windows.
campaign-enqueue	Staff JWT	Normalises phones and enqueues campaign jobs.
campaign-worker	CRON_SECRET	Claims jobs and triggers dials.
campaign-control	Staff JWT	Pause, resume, cancel, retry; audited.
retry-campaign-calls	CRON_SECRET or staff	Re-attempts failed jobs per retry policy.
reconcile-call-status	CRON_SECRET	Reconciles stuck calls against the carrier.
smart-call-routing	Staff JWT	Scores agents by skills, proficiency and performance.
analyze-call-sentiment	Internal	Sentiment per transcript turn.
transcribe-call / voice-to-text	Staff JWT	Chunked transcription with a payload cap.
officer-chat / ai-policy-notes	Authenticated	Assistants grounded in the knowledge base.
send-agent-invitation	Staff JWT	Server forces invitedBy = caller identity.
send-escalation-notification	Staff JWT	Recipients read from escalation_settings only.
send-appointment-email / send-payment-email	Internal	Transactional email.
manage-agents / manage-app-secrets	Admin JWT	Administrative CRUD.
twilio-recent-calls / twilio-verify	Staff JWT	Telephony diagnostics.
suggest-tag	Staff JWT	Heuristic plus AI tag suggestions.
sample-data	Staff JWT	Seeds demo data.
gdpr-export / gdpr-delete	Admin JWT	Privacy operations.

Function	Auth	Purpose
realtime-chat	Authenticated	Relay for the live chat panel.

4.1 Error handling contract

Functions log the full error server-side and return a safe message to the client.

```
try { /* work */ }
catch (e) {
  console.error('fn-name failed', e);
  return new Response(JSON.stringify({ error: 'An unexpected error occurred' }),
    { status: 500, headers: corsJson });
}
```

4.2 Webhook security

Inbound carrier callbacks validate the X-Twilio-Signature header using the auth token secret, with replay protection keyed on CallSid plus CallStatus.

5. Performance architecture

5.1 Index strategy

- twilio_call_sid — webhook lookups.
- campaign_id + created_at desc — campaign dashboards.
- call_status + created_at desc — live queues.
- agent_email + created_at desc — agent performance.
- phone_number — fallback matching.
- client_id + created_at desc — client-level queries.

5.2 Event log and partitioning

- The status webhook performs a pure insert into outbound_call_events, range-partitioned by month.
- process-call-events drains batches on a short cron interval.
- Call ids resolve via twilio_call_sid or parent_sid, then a single update applies the final state.
- create_outbound_call_events_partition() pre-creates next month's partition and enables RLS.

```
select public.process_outbound_call_events(500);
```

5.3 Load test results (k6)

Scenario	Load	p95	Result
Campaign execution	200 RPS, 5m	220 ms	98.4% OK, DB CPU 55%.
Concurrent dialing	500 active calls	n/a	Stable; carrier quota is the cap.
Webhook processing	1200 RPS	85 ms	100% OK after the event-log redesign.
Dashboard traffic	200 vUsers, 10m	180 ms	99.9% OK; indexes cut p95 sixfold.

6. Security

- Roles live only in user_roles and are checked through security-definer helpers.
- RLS on every public table and every partition, with explicit GRANTs.
- Execute on security-definer routines is revoked from public and anon; reporting routines are service-role only.
- Materialised views are not reachable through the API.
- Edge functions classify auth as carrier-signed, authenticated, staff JWT, admin JWT or CRON_SECRET.
- Service-role credentials are never used on the client.
- Generic client errors, full server-side logging, rate limiting on expensive endpoints.
- Audit entries for role changes, privacy operations, secret reads and escalations.
- Auth redirects pinned to https://elcareconnect.lovable.app; sign-out clears client state and returns to '/'.

7. Realtime and presence

Realtime broadcast carries live transcription, queue and transfer changes, the supervisor view and officer-chat streaming. useAgentPresence heartbeats while the tab is open, marks a stale session as new on sign-in, and writes an offline state with cleared timers on sign-out or tab close (a keepalive request covers the unload path).

8. WebRTC calling

useWebRTCCall creates one peer connection per call. SDP and ICE exchange rides a Realtime channel scoped to the call id; ICE servers are fetched from a signed edge function.

9. AI integrations

- Officer Chat and AI policy notes run through the Lovable AI Gateway.
- Tag suggestions combine a heuristic pass with an AI fallback.
- Sentiment is computed per transcript turn and stored with the transcription.
- Transcription is chunked to stay inside the payload cap.

10. Local development

```
bun install
bun run dev            # vite on :8080
bunx vitest run        # unit tests
bunx tsgo --noEmit     # typecheck
bun run docs:build     # regenerate the developer PDF

# never edit src/integrations/supabase/{client,types}.ts (auto-generated)
```

11. Documentation pipeline

- src/data/documentation.ts is the in-app documentation content, rendered by src/pages/Documentation.tsx with role-filtered chapters and search.
- scripts/docs/base_guides.py regenerates the two base PDFs in Care Connect branding.
- scripts/docs/content.mjs holds living sections appended to the developer PDF; the Vite plugin regenerates the PDF whenever those sources change.

12. Migrations

Every create table in public is followed in the same migration by GRANTs, RLS enable and policies. Backfills that add ownership columns must populate them before the new policies are enforced.

13. Observability

- error_events and system_health_metrics capture failures and health snapshots.
- audit_log records role changes, secret reads and privacy operations.
- Cloud function logs are the primary live debugging surface.

14. Testing

- Unit tests with vitest, including phone normalisation and shared function helpers.
- Load tests with k6 under load-tests/scripts.

15. Deployment

Publish from Lovable; custom domains map to the published URL. Cron schedules for process-call-events, campaign-worker, reconcile-call-status and retry-campaign-calls are configured in Lovable Cloud, and CRON_SECRET protects every cron-triggered endpoint.

16. Extending the platform

- New campaign type: insert into campaign_types, add IVR options, deploy.
- New product type: add it in Setup → Catalog so campaign forms pick it up automatically.
- New language: add the supported language, upload audio, add translation strings.
- New role: extend the role enum, add policies, update route guards.
- New AI feature: use the AI gateway; never embed third-party keys in the client.

17. Coding conventions

- Semantic Tailwind tokens only; no hardcoded colour utilities.
- Long forms split into tabs so action buttons stay visible.
- Officer chat rendered as a floating, non-modal side panel.
- Collapsible sidebar; scroll to top on every navigation.

18. Appendix: useful RPCs

RPC	Returns	Used for
has_role(uid, role)	boolean	RLS policies.
is_admin() / is_staff() / is_supervisor_or_admin()	boolean	Edge function gating.
process_outbound_call_events(limit)	integer	Drainer cron.
create_outbound_call_events_partition(date)	void	Monthly partition creation.